Exerciții — Strategy, Observer, Factory Method, Decorator

Caiet de practică · Java · hint-free

Cum lucrezi cu caietul ăsta

Sunt 9 exerciții, gradate (**[ușor]**, **[mediu]**, **[provocare]**). Nu sunt toate la fel — citește tabelul înainte să începi:

Partea	Exerciții	Cum lucrezi	Folderul
I — Strategy	S1, S2, S3	**hint-free**: primești problema, contractul și ce trebuie să iasă; clasele, contextul și testul le proiectezi tu	**ți-l faci tu**: `ex7` (S1), `ex8` (S2 + S3)
II — Observer	O1, O2, O3	la fel	**ți-l faci tu**: `ex9`, `ex10`, `ex11`
III — Factory Method	F1	**ghidat**, cerința completă e în `ex5/Cerinta.pdf`	există deja: `ex5/`
IV — Decorator	D1	ghidat, în `ex6/Cerinta.pdf`	există deja: `ex6/`
	D2	reparație în cod scris de tine	`ex1/` și `ex8/`

Când: Partea I după `ex3`, Partea II după `ex4`, D2 după `ex6`. F1 și D1 sunt chiar `ex5` și `ex6`; aici găsești doar rezumatul lor.

Reguli valabile peste tot:

- Fără `instanceof`, `getClass()`. Fără `switch` / `if` pe tip în context/subiect.
- `List` / `ArrayList` sunt permise unde ții observatori; fără `Map`.
- Mesajele de eroare în engleză.
- Fiecare folder nou are clasele în `models/` și un `TestareN` cu `public static void ruleaza()`, pe care îl adaugi tu în `Main`, după celelalte.

„Gata când" e la finalul fiecărui exercițiu: dacă rularea arată acolo, ai terminat. Teoria e în `TEORIE.pdf`. Recitește-o dacă te blochezi, dar încearcă întâi singur.

Partea I — Strategy

> Reține forma: o familie de algoritmi interschimbabili în spatele unui contract; contextul ține unul și îl întreabă, fără să știe care e.

S1 — Validator de parolă [ușor] · `ex7`

Context. Un câmp de parolă verifică dacă o parolă respectă o **politică**. Politica se poate schimba (site-uri diferite, cerințe diferite), fără să rescrii câmpul.

```
public interface PoliticaParola {
    String getNume();
    boolean esteValida(String parola);
}
```

Clasă	Regula
`PoliticaSimpla`	minim 6 caractere
`PoliticaMedie`	minim 8 caractere **și** cel puțin o cifră
`PoliticaPuternica`	minim 8 caractere, o cifră, o literă mare **și** un caracter care nu e literă sau cifră
`CampParola` (Context)	ține politica curentă; spune dacă o parolă trece; politica se poate schimba

Test. Parola `abc123` verificată cu toate 3 politicile; apoi `Abc123!x`. Afișează numele politicii și rezultatul.

Gata când aceeași parolă e acceptată de politica simplă și respinsă de cea puternică doar schimbând strategia, fără niciun `if` pe „ce fel de politică” în `CampParola`.

S2 — Comision de vânzare [mediu] · `ex8`

Context. La o vânzare se calculează comisionul agentului. Firma schimbă des modul de calcul; nu vrei să atingi clasa `Vanzare` de fiecare dată.

```
public interface Comision {
    String getNume();
    double calculeaza(double valoareVanzare);
}
```

Clasă	Formula
`ComisionFix`	o sumă fixă, dată în constructor (ex: 50), indiferent de valoare
`ComisionProcent`	un procent din valoare, dat în constructor (ex: 10%)
`ComisionPePraguri`	5% pentru partea până la 1000, 10% pentru cât depășește 1000
`Vanzare` (Context)	ține valoarea și comisionul curent; calculează suma; comisionul se poate schimba

Test. O vânzare de `1500`, calculată cu toate 3 strategiile. Verifică de mână `ComisionPePraguri`: `5% · 1000 + 10% · 500 = 100`.

Gata când cele 3 strategii dau 3 valori diferite pe aceeași vânzare, iar `ComisionPePraguri` dă `100.0` la `1500`.

S3 — Comision compus [provocare] · tot în `ex8`

Context. Extinzi S2. Vrei să aplici o limită PESTE un comision deja calculat — și eventual încă o ajustare peste aceea.

Ce construiești. `ComisionCuPlafon` primește în constructor **alt** `Comision` și o valoare maximă, și întoarce minimul dintre comisionul dinăuntru și plafon. `ComisionCuBonus` primește alt `Comision` și adaugă o sumă fixă.

Test. Plafon 200 peste un procent de 20%, pe o vânzare de `1500` → `min(300, 200) = 200`. Apoi un bonus de 50 peste rezultat → `250`.

Gata când poți înlănțui comisioane care se împachetează unul pe altul și rezultatul se calculează corect, dinăuntru spre afară.

Întrebare. O strategie care primește altă strategie de același tip și o „îmbunătățește”… îți sună cunoscut din bonusul de la `ex1`? Ăsta e un **Decorator** deghizat (lecția 4).

Partea II — Observer

> Reține forma: o sursă își schimbă starea și îi anunță pe toți abonații, fără să știe cine sunt sau ce fac. Nu așteaptă niciun răspuns înapoi.

O1 — Canal cu abonați [ușor] · `ex9`

Context. Un canal publică videoclipuri. Când apare unul nou, toți abonații sunt anunțați — fiecare în felul lui.

```
public interface Abonat {
    void notifica(String titluVideo);
}
```

Clasă	`La notifica("X")`
`AbonatEmail`	`[EMAIL] Video nou: X`
`AbonatPush`	`[PUSH] X`
`Canal` (Subject)	ține abonații; se pot abona și dezabona; publicarea unui video îi anunță pe toți

Gata când o publicare declanșează reacția tuturor abonaților, iar după o dezabonare cel scos nu mai primește nimic.

Întrebare. Scrie un abonat care se dezabonează singur, din interiorul lui `notifica`, și pune-l primul din trei. Rulează. Ce excepție apare și de ce?

O2 — Cont bancar [mediu] · `ex10`

Context. La fiecare tranzacție pe cont reacționează mai mulți observatori — dar unul reacționează **doar uneori**.

```
public interface ObservatorCont {
    void actualizeaza(double soldNou);
}
```

Clasă	Reacția
`NotificatorSms`	`[SMS] Sold nou: X` la fiecare schimbare
`JurnalAudit`	`[AUDIT] Sold inregistrat: X` la fiecare schimbare
`AlertaSoldMic`	`[ALERTA] Sold sub prag: X` **numai** dacă soldul nou e sub prag (pragul vine din constructor); altfel tace
`Cont` (Subject)	ține soldul și observatorii; depunerea și retragerea schimbă soldul și îi anunță

Test. Pornești de la `1000`, prag `100`. Retragi `950` (sold 50 → alerta se declanșează), apoi depui `500` (sold 550 → alerta tace). O retragere care ar duce soldul sub 0 aruncă `IllegalStateException`.

Gata când SMS-ul și auditul reacționează la fiecare mișcare, dar alerta apare **doar** când soldul scade sub prag.

O3 — Licitație cu auto-licitator [provocare] · `ex11`

Context. La o licitație, fiecare ofertă nouă îi anunță pe toți participanții cu noul preț. Un participant special re-licitează automat dacă e depășit.

```
public interface Participant {
    void ofertaNoua(double pretCurent);
}
```

Clasă	Reacția
`Spectator`	`[SPECTATOR] Pret curent: X`
`AutoLicitator`	are un preț maxim (din constructor); dacă prețul curent e sub el, plasează o ofertă nouă de `pretCurent + 10` **înapoi în licitație**
`Licitatie` (Subject)	ține prețul curent și participanții; acceptă doar oferte mai mari decât prețul curent (altfel `IllegalArgumentException`), setează prețul și îi anunță pe toți

Atenție la bucla de feedback. `AutoLicitator` licitează din interiorul lui `ofertaNoua`, deci o ofertă declanșează alte oferte. Ce oprește lanțul? Ai grijă ca licitatorul să nu se supraliciteze pe sine.

Gata când o singură ofertă de la un spectator declanșează o „bătălie” automată care se oprește singură la prețul maxim, și vezi în ieșire cum urcă prețul pas cu pas.

Întrebare. La O2 observatorii doar *reacționau*. La O3 un observator *declanșează o nouă schimbare în subiect*. Ce risc apare și de ce trebuie o condiție de oprire?

Partea III — Factory Method

> Reține forma: `new` e o decizie luată când scrii codul, nu când îl rulezi. Dacă vrei ca *clasa* să fie aleasă la rulare, muți `new`-ul într-un obiect care poate fi ales la rulare.

F1 — Încărcarea utilizatorilor [mediu] · `ex5`

Exercițiul ăsta e **ghidat**: scheletul e în `ex5/`, cerința completă în `ex5/Cerinta.pdf`.

Pe scurt: linii de forma `STUDENT,Ana,Popescu,anul 2` devin obiecte `Student` sau `Profesor`. `IncarcatorUtilizatori` nu are voie să conțină niciun `new Student`, niciun `switch` pe tip și niciun cast; singura comparație permisă e între **două șiruri de date**.

Gata când adaugi `ADMIN` scriind doar o clasă-model și o fabrică, fără să modifici nicio linie din încărcător.

Întrebare. De ce nu te salvează polimorfismul aici, deși `descriere()` e suprascrisă? Răspunde în scris, două rânduri. (E exact ce se întâmplă cu `getRol()` în `UserRepository.loadData()` din online-school.)

Partea IV — Decorator

> Reține forma: o clasă care semnează contractul **și** ține înăuntru un alt obiect cu același contract. Îl cheamă și îi ajustează intrarea sau rezultatul. Se stivuiesc, iar ordinea contează.

D1 — Notificări împachetate [mediu] · `ex6`

Ghidat: scheletul e în `ex6/`, cerința completă în `ex6/Cerinta.pdf`.

Pe scurt: `Notificator` cu `NotificatorEmail` și `NotificatorSms` (SMS-ul aruncă peste 30 de caractere). Peste ele scrii trei decoratori — `CuPrefix`, `CuTrunchiere`, `CuJurnal` — care se pot combina în orice ordine.

Gata când același mesaj lung, trimis prin `CuJurnal(CuTrunchiere(30, sms))` și prin `CuTrunchiere(30, CuJurnal(sms))`, dă jurnale diferite: în primul caz jurnalul vede mesajul întreg, în al doilea deja tăiat — și poți explica de ce.

D2 — Numele lanțului [ușor, dar obligatoriu] · `ex1/`, `ex8/`

Nu e un exercițiu nou, e o reparație în cod pe care l-ai scris deja. Deschide `LivrareCuReducere` (bonusul din `ex1`), `ComisionCuPlafon` și `ComisionCuBonus` (S3). Toate trei sunt decoratori. Dacă vreunul își întoarce un nume fix, de tipul `return "Cu bonus";`, își pierde identitatea a ce împachetează. Compune-l din cel dinăuntru, ca la `ex6`.

Gata când testul de la S3 afișează `Procent + plafon + bonus: 250.0` în loc de `Cu bonus: 250.0`, iar din ieșire poți citi tot lanțul fără să deschizi codul.

Întrebare. Ce se întâmplă cu un bonus negativ? Validezi plafonul în constructor, dar și bonusul? Clasele-frați scrise în aceeași ședință trebuie să se apere la fel.